Multi-Class Text Classification: A Comparison of Word Representations and ML/NN Models

Imran Rahman
Computer Science and Engineering
Brac University
22201351
imran.rahman.imran@g.bracu.ac.bd

Nabiha Kawsar
Computer Science and Engineering
Brac University
22201609
nabiha.kawsar@g.bracu.ac.bd

Faizur Salekin Rayat
Computer Science and Engineering
Brac University
22299271
faizur.salekin.rayat@g.bracu.ac.bd

Abstract*—Multi-class text classification is a common task in Natural Language Processing (NLP) with applications such as question categorization, information retrieval, and content moderation. This project compares traditional machine learning and neural network approaches for multi-class text classification using TF-IDF and Skip-gram word representations on a large dataset of over 93,000 samples across ten categories. After extensive EDA and preprocessing, models were evaluated using accuracy, macro F1-score, confusion matrices, and classification reports. Results show that Logistic Regression with TF-IDF provides a strong baseline, while Bidirectional GRU with Skip-gram embeddings delivers the best overall neural network performance, highlighting the trade-offs between sparse and distributed text representations.

Keywords*—Natural Language Processing, Machine Learning, Neural Network, Skip-gram, Text classification, LSTM, GRU, RNN

I. Introduction

Text classification aims to automatically assign predefined labels to textual documents. With the rapid growth of online text data, effective and scalable text classification methods have become increasingly important. Traditional machine learning approaches such as Logistic Regression, Naive Bayes etc. rely on sparse representations such as bag-of-words or TF-IDF, while neural network models utilize distributed word embeddings to capture semantic and contextual information.

The goal of this project is to systematically compare word representation techniques and classification models for multi-class text classification. Specifically, we analyze the performance of TF-IDF and Skip-gram representations when combined with both classical machine learning models (Logistic Regression and Deep Neural Network) and various neural network-based architectures. By conducting controlled experiments and extensive evaluation, this work provides insights into how representation choice and model complexity influence classification performance.

II. Methodology

A. Dataset Description

The dataset consists of question-answer style text in HTML format categorized into ten distinct classes, including Business & Finance, Health, Sports, Science & Mathematics, and Society & Culture. The training set contains 93,333 samples, while the test set contains 59,999 samples. Each sample consists of 2 columns, a raw HTML-formatted text labeled as "QA-Test" and a corresponding class label labeled as "class". The dataset is well-balanced across all classes.

B. Exploratory Data Analysis (EDA)

After loading the training dataset using Panda's dataframe, our exploratory data analysis revealed the following:

- There are no missing values in text or labels.
- Each class contains between 9,206–9,468 samples in the training set.
- The average document length is approximately 636 characters, with a maximum length exceeding 8,000 characters.
- Token length distribution shows significant variance, motivating sequence truncation for neural models.
- These findings guided preprocessing decisions and model design choices.

C. Preprocessing

The following preprocessing steps were applied:

- Removal of HTML tags using BeautifulSoup's HTML Parser
- Lowercasing all texts to avoid case-sensitivity
- Removal of all non-alphabetic characters including numbers and special characters if any
- Tokenization using *nltk.word_tokenize*
- Stopword removal for TF-IDF-based models
- For Skip-gram-based neural models, stopwords were retained to preserve syntactic structure. Cleaned tokens were used for both feature extraction and embedding training.

D. Word Representation Techniques

1) TF-IDF

TF-IDF vectors were generated using unigrams and bigrams with a maximum vocabulary size of 50,000 features. Sublinear term frequency scaling and document frequency thresholds were applied. TF-IDF representations were used with Logistic Regression as our classic Machine Learning model as well as a Deep Neural Network.

2) Skip-gram Embeddings

Skip-gram Word2Vec model with a vector size of 100 was trained on the training corpus. The embeddings were used in 7 neural network models, which are Deep Neural Network, SimpleRNN, GRU, LSTM, Bidirectional SimpleRNN, Bidirectional GRU and Bidirectional LSTM.

E. Model Architectures and Hyperparameter Tuning

All models were tuned using validation performance. GridSearchCV was used for Logistic Regression, in which,

the hyperparameters that were tuned are *C (inverse regularity strength)* and solver *(liblinear vs lbfgs)*.
As for the Neural Network based models, Keras Tuner (Random Search) was employed. Hyperparameters such as learning rate, number of hidden units, dropout rate, and batch size were optimized. Early stopping was applied to prevent overfitting.

III. Results and Analysis

A. Evaluation Metrics

All 9 models were evaluated on the testing data based on the following metrics:

- Accuracy
- Macro F1-score
- Confusion matrix
- Classification report

Built-in functions from scikit-learn were used to generate reports of these metrics.

B. Performance Comparison

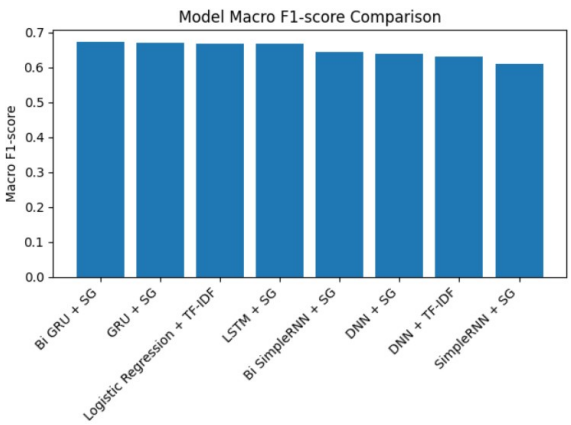
After running all the models with hyperparameter tuning, the following results were found based on the best hyperparameters.

Model	Accuracy	F1 Score
Bi GRU + SG	0.675195	0.672141
GRU + SG	0.675195	0.669585
Logistic Regression + TF-IDF	0.670411	0.668081
LSTM + SG	0.667778	0.665912
Bi GRU + SG	0.661178	0.649198
Bi SimpleRNN + SG	0.645361	0.644038
DNN + SG	0.648777	0.638921
DNN + TF-IDF	0.636494	0.630799
SimpleRNN + SG	0.609327	0.608623

Table 1: Accuracy & F1 comparison among all models

The following bar-charts compare the acuracy and f1 *scores between all models.*

Figure 1: Accuracy comparison



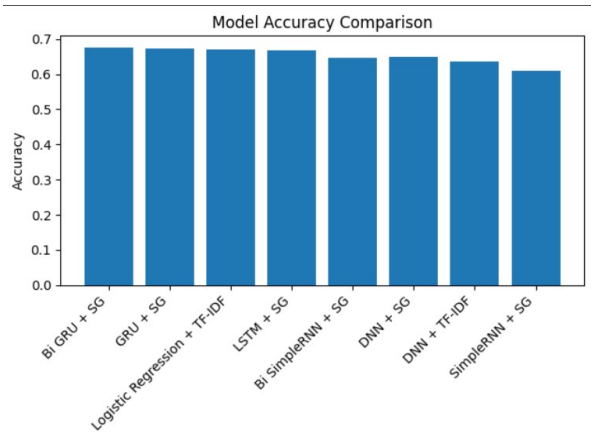


Figure 2: F1-Score comparison

The graphs were plotted using matplotlib based on their respective scores

C. Discussion

The results show that between the two TF-IDF based models, Logistic Regression with provides a strong baseline, achieving a macro F1-score of 0.668. This demonstrates that linear models with sparse representations remain effective for large, well-structured datasets.

Among the neural models, and subsequently all the nine models, Bidirectional GRU with Skip-gram embeddings achieved the best overall performance, with an accuracy of 0.675 and f1-score of 0.672, outperforming both unidirectional recurrent models and other bidirectional variants. SimpleRNN-based model performed the worst, with an accuracy of 0.609, likely due to their inability to capture long-term dependencies, which other models such as LSTM and GRU can capoture and utilize. Confusion matrix analysis indicates that misclassifications frequently occur between semantically related categories such as Education & Reference and Society & Culture.

#	**Model**	**Accuracy**	**Macro F1**
1	Bi GRU + SG	0.675195	0.672141
2	Logistic Regression + TF-IDF	0.670411	0.668081

Table 2: Best performing models in from both ML based and NN based solutions based on F1-score and accuracy

IV. Conclusion

This project provided a comprehensive comparison of word representation techniques and classification models for multi-class text classification. The experimental results show that although neural network models with Skip-gram embeddings achieve strong performance, classical machine learning approaches using TF-IDF remain highly competitive, coming in 3rd in our overall analysis. Among the neural models, the Bidirectional GRU delivered the best results, underscoring the importance of capturing contextual information in a given text.

However, the study has several limitations. The dataset size, while substantial for experimentation, is relatively small compared to real-world industrial-scale datasets. Additionally, hardware constraints led to long training times and required reducing certain hyperparameters, such as using max_features = 3000 in TF-IDF + DNN instead of 50000, to prevent system instability. Future work could address these limitations by leveraging more powerful hardware and larger datasets. Moreover, advanced models such as transformer-based

architectures with contextual embeddings can be implemented to further improve classification performance.

REFERENCES

[1] "API Reference," scikit-learn. https://scikit-learn.org/stable/api/index.html

[2] TensorFlow, "API documentation | tensorflow core v2.4.1," TensorFlow, Sep. 30, 2024. https://www.tensorflow.org/api_docs

[3] NLTK, "Natural Language Toolkit — NLTK 3.4.4 documentation," Nltk.org, 2009. https://www.nltk.org/

[4] K. Team, "Keras documentation: Keras API reference," keras.io. https://keras.io/api/